HW 2-B: OWASP Juice Shop / OWASP Top 10

CS 477 — Information Security

Student: Owen Zhang
GitHub: owenzhang04 • **Date:** Oct 3, 2026
AI Disclosure: I used Gemini 3.8 Flash High on Antigravity

PART 1: SECURE FEATURE DESIGN

Write-up: ~110 words

In OWASP Juice Shop, three critical vulnerabilities were identified: (1) **SQL Injection (SQLi)** in the authentication query via `' OR 1=1 --`, granting unauthorized administrative access; (2) **Cross-Site Scripting (XSS)** in search queries and profile forms reflecting unsanitized markup through dangerous sinks like `innerHTML`; and (3) **Broken Authentication / Insecure Password Storage**, storing unsalted MD5 hashes vulnerable to offline rainbow table lookup.

⚠ 1. SQL Injection

Exploit: Tautology `' OR 1=1 --` injects boolean logic into SQL queries.
Mitigation: Parameterized statements (prepared queries) bind inputs as literal parameters, preventing query alteration.

⚠ 2. Cross-Site Scripting

Exploit: Script payload `<script>` executes in user context.
Mitigation: Contextual output encoding and strict `textContent` DOM sinks, enforced by Content Security Policy.

⚠ 3. Insecure Passwords

Exploit: Unsalted fast hashes cracked via precomputed lookup tables.
Mitigation: Adaptive salted hashing with `bcrypt` (work factor ≥ 12) to resist offline brute force.

Secure Password Handling Implementation (Node.js `bcrypt`):

```
const bcrypt = require('bcryptjs');

// 1. User Registration: Hash password with a unique, cryptographically random salt (cost: 12)
const SALT_ROUNDS = 12;
const passwordHash = await bcrypt.hash(plainPassword, SALT_ROUNDS);

// 2. User Authentication: Timing-safe verification against stored hash
const isPasswordValid = await bcrypt.compare(submittedPassword, storedHash);
```

PART 2: FRONT-END FORM IMPLEMENTATION & DUAL VALIDATION

Write-up: ~105 words

Public GitHub Repository: https://github.com/owenzhang04/owasp-juice-shop-login

MIT Licensed • Node.js

I developed a responsive login interface mirroring OWASP Juice Shop's aesthetic, incorporating email, password, and visibility toggling. The architecture enforces **dual-layer validation**:

- **Client-Side Validation (`public/app.js`):** Intercepts form submissions to block blank inputs, verifies that the email contains the mandatory `"@"` symbol, and enforces a minimum password length of 8 characters (`password.length ≥ 8`). UI errors and alerts are populated exclusively via `textContent` to prevent DOM-based XSS.
- **Server-Side Validation (`server.js`):** Implements zero-trust defense by re-validating non-emptiness, RFC 5322 email syntax, and password length. Authentication uses parameterized lookup logic and salted key derivation (`scrypt` / `bcrypt`) with constant-time equality checks (`crypto.timingSafeEqual`).

PART 3: EXPLOITATION ATTEMPT & SECURITY FINDINGS

Write-up: ~120 words

I attempted two primary attacks against the newly implemented login system: (1) **SQL Injection (SQLi):** Inputting `' OR 1=1 --` and `admin@juice-sh.op' --` to bypass credentials; and (2) **Cross-Site Scripting (XSS):** Injecting `<script>alert('XSS')</script>@juice-sh.op` to trigger browser code execution.

Execution Results: Both attacks failed. The client-side validator immediately blocked the SQLi payload because it lacked an `@` symbol. When bypassing the client interface via direct HTTP POST requests, the server-side validator rejected malformed inputs with `HTTP 400 Bad Request`. For the targeted email attack, the parameterized database lookup treated the string literally, returning `HTTP 401 Unauthorized` without SQL evaluation. XSS payloads were treated purely as string data in safe `textContent` sinks without script execution.

Proposed Fix & Hardening: Maintain a strict Content Security Policy (`Content-Security-Policy: default-src 'self'`) and enforce server-side RFC 5322 regex sanitization to safeguard against any bypassed client controls.

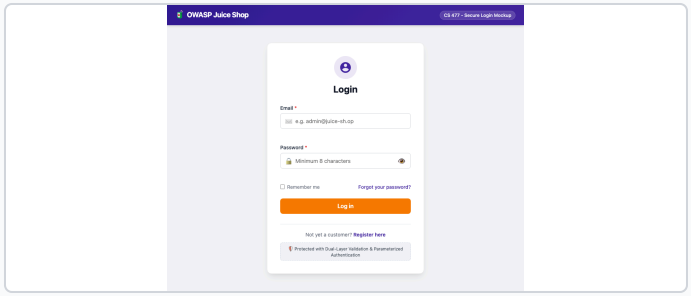


Figure 1: Initial clean login form mimicking OWASP Juice Shop UI.

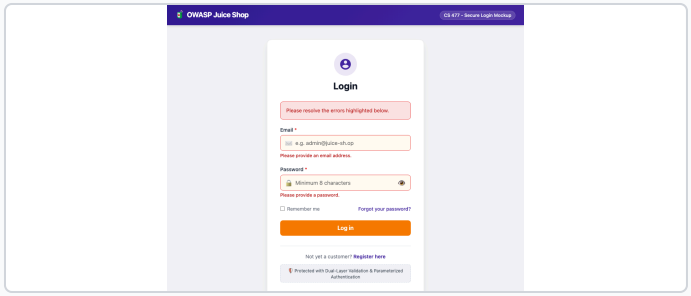


Figure 2: Client-side validation blocking empty form submissions.

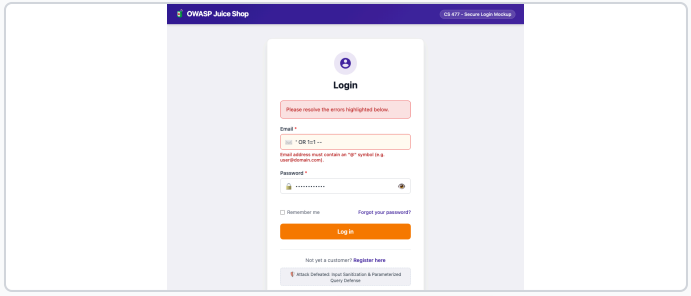


Figure 3: SQL Injection attempt (`' OR 1=1 --`) blocked by email validation.

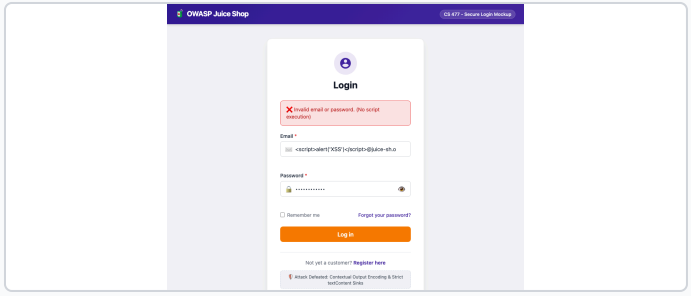


Figure 4: XSS attempt safely rendered as text without script execution.

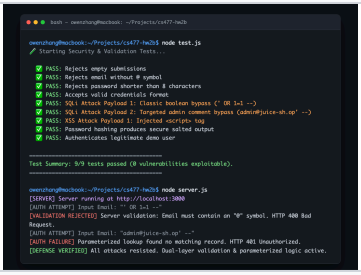


Figure 5: Automated security test suite (9/9 passed) & backend server-side rejection logs.